

Keywords: sparse attention • KV cache compression • end-to-end training • long-context LLM • differentiable gating

SAS: Simple Attention Sparsification via End-to-End Optimization

Hard Top-K Blocks Gradient Flow in Trainable Sparse Attention — Continuous Log-Space Gates Restore Differentiability and Outperform Baselines Under Tight KV Budgets

By Zhiwei Li, Lei Zhu, Hao Gu, Xiang Hu, Yan Wang, Haitao Mi, Sirui Han, Leo Liang, Zhijiang Guo (Tencent HY LLM Frontier; HKUST)

arXiv:2609.13141

Efficient long-context inference demands reducing attention from $O(L^2)$ to sub-linear cost by retaining only a sparse subset of KV entries per query. Trainable selectors promise to learn which tokens matter for downstream tasks — but a fundamental problem has gone unaddressed: the hard Top- K gate that implements sparsification is non-differentiable, severing gradient flow from the language modeling loss back to the selector. SAS (Simple Attention Sparsification) fixes this with continuous log-space gates injected directly into the attention softmax, enabling full end-to-end training with no architectural changes.

AT A GLANCE

Problem: Hard Top- K in trainable sparse attention blocks gradient flow; selectors train on surrogate losses misaligned with language modeling.

Why it matters: Long-context LLMs need 5–10% KV budgets to fit in memory; heuristic selectors lose 5–7 points on reasoning at these budgets.

Method: Inject gate $g_i = \sigma(s_i)$ as $\log g_i$ into attention logits before softmax; normalize against mandatory current block.

Result: Closes most of the gap to full attention at 10% KV budget on reasoning, long-context, and agentic benchmarks.

Evidence: LLaMA-3-8B and 70B, Mistral-7B; MATH-500, LongBench, WebArena, SWE-bench-lite.

Why Existing Approaches Fall Short

Sparse attention methods select a subset $\mathcal{S} \subset \{1, \dots, L\}$ of KV entries per query using a lightweight selector that outputs relevance scores $s_1, \dots, s_L$, then applies:

$$\mathcal{S} = \text{Top-}K(s_1, \dots, s_L)$$

The Top- K operation has zero gradient almost everywhere: $\partial \mathcal{S} / \partial s_i = 0$. Selectors are therefore trained with surrogate losses such as cosine similarity between selector output and the query — a proxy for what matters in generation. This mismatch is the root cause of quality degradation under tight budgets.

Core Mechanism: Log-Space Gate Injection

SAS replaces the hard gate with a continuous sigmoid gate:

$$g_i = \sigma(s_i) = \frac{1}{1 + e^{-s_i}} \in (0, 1)$$

The gate is injected into the attention logit before softmax:

$$\tilde{a}_{ij} = \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} + \log g_j \right)_j$$

Adding $\log g_j$ is equivalent to multiplying the pre-softmax probability by g_j :

$$\tilde{a}_{ij} \propto \exp \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} \right) \cdot g_j$$

When $g_j \rightarrow 0$ a token is suppressed; when $g_j \rightarrow 1$ it is fully retained. Because softmax is differentiable, the gradient of the language modeling loss flows back through $\tilde{a}_{ij}$, through the gate g_j , and to the selector parameters:

$$\frac{\partial \mathcal{L}_{\text{LM}}}{\partial s_j} = \frac{\partial \mathcal{L}_{\text{LM}}}{\partial \tilde{a}_{ij}} \cdot \frac{\partial \tilde{a}_{ij}}{\partial g_j} \cdot \sigma'(s_j)$$

Technical Formulation

Gate-injected attention:

$$\tilde{\mathbf{a}}_i = \text{softmax} \left(\frac{\mathbf{q}_i K^\top}{\sqrt{d}} + \log \mathbf{g} \right), \quad \mathbf{g} = \sigma(\mathbf{s})$$

Selector: A 2-layer MLP or single attention head:

$$s_j = f_{\theta_s}(\mathbf{k}_j)$$

Training: Standard causal language modeling with gradients flowing through attention and the gate to θ_s :

$$\mathcal{L} = - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

Inference: Select top- K entries by s_j (hard), attend only to those. The selector has been trained with soft gradients.

Normalized Softmax Gates

The current block $\mathcal{C}$ (the most recent w tokens) is always retained in full. SAS normalizes historical gate scores against the attention mass consumed by $\mathcal{C}$:

$$\hat{g}_j = g_j \cdot (1 - A_{\mathcal{C}}), \quad A_{\mathcal{C}} = \sum_{j \in \mathcal{C}} \tilde{a}_{ij}, \quad j \notin \mathcal{C}$$

This calibrates the historical context budget to the remaining attention mass after the current block is satisfied, keeping sparsification levels consistent across sequence positions.

Learning or Inference Procedure

SAS is applied as a post-training fine-tuning step on a frozen or lightly-tuned backbone. The selector is initialized randomly or from heuristic scores and fine-tuned for as few as 500 steps on standard language modeling corpora. The MLP selector operates on cached keys and adds $O(L)$ overhead — negligible against $O(L^2)$ attention.

Experimental Results

Evaluated on LLaMA-3-8B, LLaMA-3-70B (long-context variants) and Mistral-7B-v0.3 at 10% KV budget:

Method	MATH-500	LongBench	WebArena
Full Attention	68.4	54.2	17.3
Quest	61.2	49.1	14.6
SnapKV	59.7	47.8	13.9
MagicPIG	63.1	50.4	15.1
SAS	66.8	53.1	16.9

SAS closes most of the gap to full attention at 10% KV budget, while prior trainable methods lose 5–7 points on MATH-500.

Ablations and Interpretation

Without log-gate (hard Top- K , end-to-end attempt): Training fails to converge; gate gradient is zero; performance equals Quest level.

Without normalized gate: Degradation of 3.2 points on LongBench; model over-attends to historical tokens and ignores the current block.

Selector depth: 2-layer MLP is sufficient; deeper selectors do not improve quality.

Budget sensitivity: SAS advantage is largest at tight budgets (5–10% KV); at 50% all methods approach full attention.

Reference

Zhiwei Li et al. **SAS: Simple Attention Sparsification via End-to-End Optimization of Context Ranking**. arXiv:2609.13141, September 2026. <https://arxiv.org/abs/2609.13141>

Keywords: recurrent neural networks • depth vs. width • compute allocation • sequence modeling • neural scaling

Temporal Recurrence Favors Fewer Layers

Recurrent Models Peak at 2–4 Layers While Non-Recurrent Models Keep Improving Up to 32 — Temporal Depth and Within-Step Depth Provide Overlapping Computational Benefits

By Ivan Anokhin, Johan Obando-Ceron, Irina Rish, Sebastian Risi (Mila / Université de Montréal; University of Copenhagen)

arXiv:2609.12531

Deep learning lore holds that depth is essential for model expressivity. This intuition was developed for non-recurrent models that process all sequence positions in parallel and must instantiate all computation simultaneously. Recurrent architectures add a qualitatively different computation axis: temporal recurrence, which accumulates processing across sequence positions step by step. This paper asks: when temporal recurrence is available, how much within-step depth is still needed? The answer is surprisingly little — and the difference has direct implications for how recurrent models should be scaled.

AT A GLANCE

Problem: How should compute be allocated across within-step depth, expert width, and parallel experts for recurrent vs. non-recurrent models?

Why it matters: Miscalibrated depth wastes compute that could go to width; the right allocation changes at optimal recurrent model design.

Finding: Recurrent models peak at 2–4 layers; non-recurrent models keep improving through 8–32 layers.

Explanation: Temporal recurrence and within-step depth both allow later computation to condition on earlier outputs — recurrence makes depth partly redundant.

Evidence: Sokoban planning and FineWeb language modeling; matched FLOPs across depth, width, expert-count axes.

The Big Picture

A non-recurrent Transformer of depth D' processes each token through D' layers applied once in parallel. A recurrent model of within-step depth D applied over T sequence positions uses $D \cdot T$ total layer applications. Under matched FLOPs, the question is: for a fixed $D \cdot T$ budget, should D be large (deep per-step, fewer steps of computation) or small (shallow per-step, many recurrent

steps)?

The answer the authors find empirically: small D with compensating width is consistently better for recurrent models, while large D is better for non-recurrent models. This suggests that within-step depth and temporal depth are not fully orthogonal dimensions — they provide partly overlapping computational benefits.

Technical Formulation

Let $\mathbf{h}_t^{(0)} = \mathbf{x}_t$ be the input at position t . The recurrent forward pass applies D within-step layers:

$$\mathbf{h}_t^{(d)} = f_d(\mathbf{h}_t^{(d-1)}, \mathbf{h}_{t-1}^{(D)}), \quad d = 1, \dots, D$$

where $\mathbf{h}_{t-1}^{(D)}$ is the hidden state from the previous position, and weights are shared across positions (weight tying across time). For E parallel experts:

$$\mathbf{h}_t^{(d)} = \sum_{e=1}^E \pi_e(\mathbf{h}_t^{(d-1)}) \cdot f_d^{(e)}(\mathbf{h}_t^{(d-1)})$$

Compute matching: When depth is reduced from D to D/k , the hidden dimension is increased by $\sqrt{k}$ to hold parameters constant (quadratic scaling of attention/MLP with hidden dim).

Arithmetic intensity: Total layer applications per sequence equal $D \cdot T$ (recurrent) vs. D' (non-recurrent, applied once to all positions in parallel). FLOPs are matched across configurations.

What the Results Show

Sokoban Planning (Success Rate, %)

Depth D	Recurrent	Non-Recurrent
1	44.2	31.8
2	67.9	49.3
4	71.3	58.7
8	70.1	66.4
16	68.4	70.8
32	66.7	72.1

Recurrent models peak at $D = 4$; non-recurrent models keep improving through $D = 32$. At the recurrent optimum ($D = 4$), the recurrent model outperforms the non-recurrent model by 12.6 points at the same FLOPs.

FineWeb Language Modeling (Validation Perplexity, ↓)

Depth D	Recurrent	Non-Recurrent
2	18.7	22.4
4	17.3	20.1
8	17.8	18.6
16	18.4	17.9
32	19.1	17.2

Consistent finding: recurrent optimal at $D = 4$; non-recurrent at $D = 16$ –32.

Ablations and Interpretation

Width compensation: Reducing recurrent depth from 8 to 2 layers while increasing hidden dimension to match FLOPs recovers 97% of 8-layer recurrent performance on both tasks. Width is a suitable substitute for depth in recurrent networks.

Expert count: Increasing parallel experts from 1 to 4 (same compute via width reduction) provides consistent modest gains for both architectures, orthogonal to the depth effect.

Shared vs. unshared weights: Unshared weights slightly improve non-recurrent baselines but do not change the qualitative finding for recurrent models.

Sequence length effect: Longer sequences amplify the advantage of shallow recurrent models over deep ones, consistent with the interpretation that temporal depth substitutes for within-step depth; more recurrent steps make additional layers more redundant.

Reference

Ivan Anokhin, Johan Obando-Ceron, Irina Rish, Sebastian Risi. **Temporal Recurrence Favors Fewer Layers.** arXiv:2609.12531, September 2026. <https://arxiv.org/abs/2609.12531>

Keywords: heterogeneous hardware • subquadratic attention
• disaggregation • energy efficiency • LLM serving

Rethinking Heterogeneous System Disaggregation for Subquadratic Attention

Prefill-Decode Disaggregation Misses Within-Decode Arithmetic Intensity Heterogeneity — Fine-Grained SQD Delivers 31–56% Tokens-per-Joule Gains

By Arya Tschand, Yaosheng Fu, Vikram Sharma Mailthody, Nicolai Oswald, Po-An Tsai, Ritchie Zhao, Oreste Villa, Vijay Janapa Reddi, Karu Sankaralingam (Harvard; NVIDIA; UW-Madison)

arXiv:2609.13134

Frontier LLMs increasingly deploy subquadratic attention — sparse, linear, and sliding-window variants — to reduce the $O(L)$ KV memory footprint of long contexts. Heterogeneous hardware systems already disaggregate by phase: prefill (compute-bound) on one device, decode (memory-bound) on another. But this coarse split ignores a critical observation: within a single decode step, different operators have sharply different arithmetic intensity profiles. SQD (SubQuadratic Disaggregation) exploits this by splitting the decode step itself, routing memory-bound and compute-bound sub-phases to the hardware optimized for each.

AT A GLANCE

Problem: Prefill-decode disaggregation does not account for within-decode operator heterogeneity in models with subquadratic attention.

Why it matters: 31–56% energy efficiency gain is left on the table on frontier models (GLM 5.2, Nemotron 3 Ultra, Gemma 4 31B).

Method: SQD splits decode by sub-phase: top- k selection (memory device) vs. top- k attention + FFN (compute device) for sparse; dense layers vs. subquadratic + FFN for linear models.

Result: 53%, 31%, 56% tokens/J improvement on GLM 5.2, Nemotron 3 Ultra, Gemma 4 31B on an 8×B200 system.

Evidence: Energy and throughput measured on production model weights; ablations by disaggregation level.

The Big Picture

Full attention has arithmetic intensity $I = \text{FLOPs/bytes} \approx d/2$ (roughly compute-bound for large d). Decode batch size 1 is memory-bound ($I \approx 1$) regardless of attention type. The roofline model predicts runtime:

$$T(O) = \max\left(\frac{F}{C}, \frac{B}{\beta}\right)$$

where C (FLOP/s) and β (bytes/s) are device peaks. Routing to the device where this bound is tightest minimizes runtime and energy per token.

Why Existing Approaches Fall Short

Current disaggregation schemes split at the phase boundary: prefill (high compute intensity, batch = L) goes to GPU; decode (low intensity, batch = 1–few) goes to memory-optimized accelerator. This is appropriate for dense attention. For subquadratic attention, the decode step itself contains operators with radically different intensities:

Operator	I (FLOP/byte)	Bound
Top- k KV selection	≈ 1	Memory
Top- k attn. + FFN	$\approx 10^3$	Compute
Dense attn. layer	≈ 10	Memory
Subquad. layer + FFN	≈ 2	Memory

Routing the entire decode step to a memory-optimized device wastes compute capacity on operators that are actually compute-bound.

Core Mechanism: SQD

SQD for sparse attention. Each decode step is split into two sub-phases:

1. *Selection:* Traverse all L keys to compute relevance scores and select top- k entries. Cost: $O(L \cdot d_{\text{head}})$ reads, $O(L)$ FLOPs. $I \approx 1$ FLOP/byte $\rightarrow$ memory device.
2. *Attention + FFN:* Attend to the selected k entries and run the FFN. With $k \ll L$, KV footprint is small and static; $I \approx 10^3$ FLOP/byte $\rightarrow$ compute device.

SQD for linear/sliding-window attention. Split by layer type:

1. *Dense attention layers:* Memory-bound at decode batch 1 $\rightarrow$ memory device.
2. *Subquadratic layers + FFN:* Linear attention state update is compute-bound $\rightarrow$ compute device.

Communication cost: The two sub-phases exchange activation tensors of size $O(d)$ per token per layer — small compared with $O(L \cdot d)$ KV data transfer.

Technical Formulation

The **ridge point** of a device is:

$$I^* = \frac{C}{\beta}$$

On B200: $C = 4.5 \times 10^{15}$ FP8 FLOP/s, $\beta = 8 \times 10^{12}$ bytes/s, so $I^* \approx 562$ FLOP/byte.

Routing criterion: Assign operator O to the memory-optimized device if $I(O) < I^*$, otherwise to the compute-optimized GPU.

Energy per token: Routing each operator to its optimal device reduces average power draw per operation; the total energy per token is:

$$E = \sum_O \frac{F_O}{P_O} + \frac{B_O}{P_O^{\text{mem}}}$$

where P_O is the TDP of the assigned device.

Experimental Results

Evaluated on an 8×B200 heterogeneous system (4 compute-optimized + 4 memory-bandwidth-optimized) with production model weights:

Model	Attn. Type	Tokens/J Gain
GLM 5.2	Sparse	+53%
Nemotron 3 Ultra	Linear	+31%
Gemma 4 31B	Sliding-win	+56%

Throughput gains (tokens/s, batch = 8, seq = 32k):

Model	GPU-only	SQD	Speedup
GLM 5.2	1,840	2,710	1.47×
Gemma 4 31B	2,190	3,080	1.41×

Ablations and Interpretation

Selection-only disaggregation: Routing only the top- k selection phase accounts for 68% of the total energy gain for sparse attention models.

Without within-decode layer-type split: Routing by prefill/decode phase only reduces energy gain from 53% to 29% for GLM 5.2, quantifying the value of intra-decode disaggregation.

Communication overhead: Cross-device activation transfer adds 7–12% latency but is offset by 40–55% reduction in time-per-token on memory-bound operators.

Dense models: SQD provides no benefit when all attention layers are dense — standard prefill-decode disaggregation is already optimal in that case.

Reference

Arya Tschand et al. **Rethinking Heterogeneous System Disaggregation for Subquadratic Attention**. arXiv:2609.13134, September 2026. <https://arxiv.org/abs/2609.13134>

Keywords: motion generation • point cloud representation
• text-to-motion • animal motion • diffusion models

UniMo: Unifying Human and Animal Motion Generation

Parametric Skeletons Block Cross-Species Unification — Topology-Agnostic Point Clouds and a Dataset 102× Larger Than Any Prior Animal Corpus Enable a Single Model for All Species

By Zeyu Zhang, Zhiyuan Zhang, Siheng Wang, Yiran Wang, Danning Li, Ian Reid, Richard Hartley (University of Adelaide; Australian National University)

arXiv:2609.12342 — SIGGRAPH Asia 2026

Text-driven 3D motion generation has matured rapidly for humans, with standardized SMPL skeletons enabling a community of comparable models. Animals remain largely excluded: a dog has 17 joints, a horse 30, a bird 15 — their skeletal topologies are incompatible with each other and with SMPL, forcing per-species model training. UniMo eliminates this barrier by converting any skeleton to a fixed-size unparametric point cloud, training a single diffusion model across species, and releasing UniML3D, a dataset that is 102 times larger than any existing animal motion corpus.

AT A GLANCE

Problem: Diverse animal skeletal topologies prevent training a single motion generation model across species.

Why it matters: Per-species models are expensive; animal motion datasets are tiny (<1,500 sequences); SIGGRAPH applications demand diverse creatures.

Method: Convert parametric skeletons to $N = 256$ point clouds with dynamic sampling; train single diffusion model; IK reconstruction per species at test time.

Dataset: UniML3D: 145,907 sequences + 433,388 captions across human and animal categories.

Result: State of the art on HumanML3D, KIT-ML, and AnimalML3D simultaneously in one model.

Why Existing Approaches Fall Short

Motion generation models represent poses as joint rotation/position vectors in species-specific fixed order:

$$\mathbf{P}_{\text{human}} \in \mathbb{R}^{22 \times 3}, \quad \mathbf{P}_{\text{dog}} \in \mathbb{R}^{17 \times 3}, \quad \mathbf{P}_{\text{horse}} \in \mathbb{R}^{30 \times 3}$$

Dimension and joint semantics differ across species; concatenating these into a single input space is undefined

without topology alignment that does not exist. The result: every new species requires a separate model trained from scratch.

Core Mechanism: Point Cloud Representation

UniMo samples $N = 256$ points from the bones of any skeleton, regardless of species topology. For each bone b connecting joints j_a and j_b , n_b equally-spaced points are placed along the bone segment. The pose at time t is:

$$\mathbf{P}_t \in \mathbb{R}^{N \times 3}, \quad N = \sum_b n_b$$

This representation is topology-agnostic (always 256 points), continuous (captures limb geometry, not just endpoints), and differentiable (linear interpolation along bones).

Dynamic sampling. Bone activity at time t :

$$v_{b,t} = \|\Delta \mathbf{p}_{j_a,t}\|_2 + \|\Delta \mathbf{p}_{j_b,t}\|_2$$

Mean activity: $\bar{v}_b = \frac{1}{T} \sum_t v_{b,t}$. Point allocation:

$$n_b = \max\left(1, \left\lfloor N \cdot \frac{\bar{v}_b}{\sum_{b'} \bar{v}_{b'}} \right\rfloor\right)$$

This concentrates the N -point budget on actively moving bones, sharpening the representation of the most informative parts of motion.

Technical Formulation

Diffusion forward process:

$$\mathbf{P}_t^{(\tau)} = \sqrt{\alpha_\tau} \mathbf{P}_t^{(0)} + \sqrt{1 - \alpha_\tau} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$$

Denoising objective:

$$\mathcal{L} = \mathbb{E}_{\tau, \boldsymbol{\epsilon}} \left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{P}^{(\tau)}, \tau, \mathbf{c}) \right\|^2 \right]$$

where $\mathbf{c}$ is a CLIP text embedding and $\boldsymbol{\epsilon}_\theta$ is a Transformer denoiser with cross-attention to $\mathbf{c}$.

Inverse kinematics reconstruction: Given predicted point cloud $\hat{\mathbf{P}}_t$, recover joint rotations by minimizing:

$$\hat{\boldsymbol{\theta}}_t = \arg \min_{\boldsymbol{\theta}} \sum_b \left\| \text{FK}(\boldsymbol{\theta})_b - \hat{\mathbf{P}}_{t,b} \right\|^2 + \lambda \sum_b (\|\Delta \text{FK}_b\|_2 - \ell_b)^2$$

where FK is forward kinematics and ℓ_b is rest-pose bone length.

UniML3D Dataset

Category	Sequences	Captions
Human	128,319	384,957
Animal (quadruped)	14,231	42,693
Animal (avian)	3,357	5,738
Total	145,907	433,388

The largest prior animal dataset contains $\sim 1,432$ sequences; UniML3D is $102\times$ larger.

Experimental Results

Method	HumanML3D FID↓	KIT-ML FID↓	AnimalML3D FID↓
MDM	0.544	0.497	—
MotionDiffuse	0.630	1.954	—
MLD	0.473	0.404	—
Species-specific	—	—	2.14
UniMo	0.412	0.381	1.07

UniMo improves over MLD on both human benchmarks while being the first unified model to achieve competitive quality on animals.

Ablations and Interpretation

Without dynamic sampling (static uniform): AnimalML3D FID degrades by 0.11; most degradation on fast-moving quadruped sequences.

Without joint training on animals: AnimalML3D FID = 4.32 (near random), confirming human-only training does not generalize.

Point count N : Plateaus at $N = 256$; $N = 128$ degrades FID by 0.09 on animals; $N = 512$ shows no improvement.

IK reconstruction: Mean bone-length error of 1.2 mm ($< 0.1\%$ of body height), confirming the IK solver accurately recovers valid joint configurations from predicted point clouds.

Reference

Zeyu Zhang et al. **UniMo: Unifying Human and Animal Motion Generation**. arXiv:2609.12342, September 2026. SIGGRAPH Asia 2026. <https://arxiv.org/abs/2609.12342>

Keywords: tabular foundation models • FP8 quantization • in-context learning • attention speedup • TabPFN

Attention Quantization for Tabular Foundation Models

Tabular FM KV Matrices Scale Quadratically With Training Rows — FP8 Quantization With Test-Training Row Alignment Achieves 1.7× Speedup at Zero Accuracy Cost

By Jonas M. Kübler, Benjamin Jäger, Klemens Flöge, Noah Hollmann, Frank Hutter (University of Freiburg; ELLIS Institute Tübingen)

arXiv:2609.13031

Tabular foundation models such as TabPFN and TabICL perform in-context learning on new datasets by encoding the entire training set as a sequence of tokens and attending over all rows in a single forward pass. Their attention matrices scale quadratically with training set size — a table with 10,000 rows and 4 attention heads contains 400 million attention entries per layer, dwarfing the KV caches of generative LLMs. FP8 quantization has transformed LLM serving, but applying it to tabular FMs requires solving a unique alignment problem: the quantization error on test rows must match that on training rows, or prediction accuracy collapses.

AT A GLANCE

Problem: Tabular FM KV matrices scale $O(n^2)$ with training rows; FP8 quantization without row-group alignment degrades accuracy by 3–5 points.

Why it matters: At $n = 10,000$ training rows, FP16 attention costs 800 MB per layer; FP8 halves this and doubles bandwidth.

Method: Dynamic per-group FP8 scale calibration with test/train row alignment; custom asymmetric-tile Triton kernel.

Result: 1.7× speedup on H100 with <0.2-point accuracy change on TabPFN-v3 and TabICLv2.

Evidence: TabArena (47 datasets), BeyondArena (21 datasets); ablations on alignment, format, tile shape.

The Big Picture

In a tabular FM, all n training rows and m test rows are encoded as tokens and processed in one forward pass. The attention matrix $A \in \mathbb{R}^{(n+m) \times (n+m)}$ has $(n+m)^2$ entries per head per layer. For $n = 10,000$, $m = 1,000$, and 4 heads at FP16:

$$\text{memory} = 4 \times (1.1 \times 10^4)^2 \times 2 \text{ B} \approx 968 \text{ MB per layer}$$

This exceeds the KV cache of a 7B LLM processing a 128k-token context at the same precision. FP8 quantization halves this cost and doubles effective bandwidth utilization.

The Quantization Error Misalignment Problem

Standard FP8 quantization assigns a single scale s to a tensor X :

$$s = \frac{\|X\|_\infty}{448}, \quad \hat{X} = \text{round}(X/s) \cdot s$$

(448 is the maximum value in FP8 e4m3fn format.)

In a tabular FM, the query matrix $Q = [Q_{\text{train}}; Q_{\text{test}}]$ has two row groups with different distributions: training rows include label information in their embeddings; test rows do not. If $\|Q_{\text{train}}\|_\infty \gg \|Q_{\text{test}}\|_\infty$, then s is calibrated to training rows, and test queries use only the low end of the FP8 range — far fewer distinct values — introducing systematic bias in test attention scores.

Core Mechanism: Per-Group Dynamic Alignment

SAS applies separate dynamic scales per row group:

$$s_g = \frac{\|Q_g\|_\infty}{448}, \quad g \in \{\text{train}, \text{test}\}$$

Computed from the tensor itself at each forward pass (no calibration dataset). This equalizes relative quantization error:

$$\frac{\epsilon_q}{\|Q_g\|_\infty} \leq \frac{1}{2 \times 448} \approx 0.001$$

for both row groups, eliminating the systematic bias.

Technical Formulation

FP8 e4m3fn: 4 exponent bits, 3 mantissa bits, 1 sign bit. Representable range $[-448, 448]$; step size $s/8$ per scale.

Per-group quantized attention:

$$\hat{Q}_g = \text{round}(Q_g/s_g) \cdot s_g$$

$$A_{ij} = \text{softmax}\left(\frac{\hat{\mathbf{q}}_i \hat{\mathbf{k}}_j^\top}{\sqrt{d}}\right), \quad \mathbf{O}_i = \sum_j A_{ij} \hat{\mathbf{v}}_j$$

Alignment overhead: One pass over the input to compute per-group max values; <1% of total attention compute time.

Speedup bound: Bandwidth-bound regime (large n):

$$\text{Speedup}(n) \approx \frac{16}{8} \cdot \frac{B_{\text{FP16}}}{B_{\text{FP8}}} = 2 \times (1 - \epsilon_{\text{kernel}})$$

with kernel overhead $\epsilon_{\text{kernel}} \approx 15\%$ for the custom Triton kernel.

Experimental Results

Tested on TabPFN-v3 and TabICLv2 on an H100 GPU:

Model	Precision	Speedup	Acc. Change
TabPFN-v3	FP16	1.00×	—
TabPFN-v3	FP8 (no align)	1.65×	−3.1%
TabPFN-v3	FP8 (aligned)	1.70×	−0.1%
TabICLv2	FP16	1.00×	—
TabICLv2	FP8 (no align)	1.62×	−4.7%
TabICLv2	FP8 (aligned)	1.68×	−0.2%

Alignment eliminates 3–5 point accuracy degradation; the residual <0.2 points is within noise. Benchmarks: TabArena (47 classification datasets), BeyondArena (21 regression datasets).

Ablations and Interpretation

Without per-group scale (global scale): Accuracy drops 3.1–4.7 points — the distribution mismatch between test and training rows is the dominant failure mode.

E4m3fn vs. E5m2: E4m3fn (more mantissa bits) outperforms E5m2 (more exponent bits) because attention logit distributions are well-concentrated and benefit more from mantissa precision.

Tile shape: Custom asymmetric tiling adds 18% speedup over reusing a standard square-tile FP8 kernel, justified by the asymmetric $m \times n$ ($m \ll n$) shape of tabular FM attention.

Scaling with n : Speedup grows from $1.3\times$ at $n = 500$ to $1.9\times$ at $n = 10,000$, reflecting that larger attention matrices are more bandwidth-bound.

Reference

Jonas M. Kübler, Benjamin Jäger, Klemens Flöge, Noah Hollmann, Frank Hutter. **Attention Quantization for Tabular Foundation Models.** arXiv:2609.13031, September 2026. <https://arxiv.org/abs/2609.13031>